ENTERPRISE IAM QUALITY ASSURANCE & PROVISIONING GOVERNANCE

SAILPOINT IDENTITY SECURITY CLOUD (ISC) ACTIVE DIRECTORY PROVISIONING AUTOMATION

System Architecture, Technical Design Specification, & Exhaustive Component Reference

Document Title	System Architecture & Component Reference Manual	**Framework Version**	v2.0 (Enterprise Modular Architecture)
Target Environment	SailPoint ISC Tenant (EU-Partner28)	**Target Directory**	Active Directory (AD_Demo_Source)
Target Audience	Associate Directors, IAM Architects, QA Engineers	**Authoritative Source**	Delimited_Employees (HR Feed)
Publication Date	November 06, 2026	**Classification**	**INTERNAL USE ONLY - IAM AUDIT ARTIFACT**

EXECUTIVE MANDATE & SYSTEM OVERVIEW:
This architectural specification defines the technical design, domain structure, verification mechanics, and component hierarchy of the **SailPoint ISC Active Directory (AD) Provisioning Automation Framework**. The system standardizes end-to-end testing of identity lifecycle states, account provisioning, and access request governance without manual UI overhead. It executes programmatic REST API interactions against SailPoint ISC, enforces a 4-tier technical verification pipeline against downstream target directories, and produces executive analytical audit workbooks for leadership compliance review.

KEY ARCHITECTURAL PILLARS

Pillar	Design Principle	Operational Impact
Zero UI Overhead	100% API-driven execution via official SailPoint ISC v3 and Beta REST endpoints with OAuth2 token caching.	Replaces hours of manual browser clicks with deterministic, repeatable, scripted testing cycles.
Pluggable Core Engine	Open-Closed design decoupling ExecutionContext, BaseScenario, and ScenarioRegistry with dynamic discovery.	Zero blast-radius. New tenant scenarios or use cases drop in without modifying runner or CLI code.
Multi-Tier Verification	4-tier validation pipeline: Task Polling -> Live Identity State -> Account Status -> Policy Outcome.	Guarantees that downstream Active Directory accounts actually reflect upstream identity governance policies.

Institutional Reporting	Automated generation of 5-tab analytical Excel workbooks, executive Markdown summaries, and JSON audit logs.	Delivers zero-emoji, formal corporate artifacts ready for Associate Director and regulatory audit review.

1. HIGH-LEVEL SYSTEM ARCHITECTURE & INTEGRATION CONTEXT

The framework functions as an automated test and verification layer positioned between enterprise quality engineers and SailPoint Identity Security Cloud (ISC). It coordinates with the tenant's REST APIs, orchestrates lifecycle transitions, monitors backend task execution queues, and asserts downstream directory state.

Layer	Component	Protocol / Interface	Core Responsibilities
Presentation	Interactive CLI & MenuManager	`CLI Console (stdin/stdout)`	2-Tier categorized navigation, identity search prompts, parameter inputs, and execution verification.
Core Engine	core/ Execution Runtime	`Python In-Memory Context`	ExecutionContext runtime model, BaseScenario abstraction, dynamic auto-discovery registry, and dispatch.
Domain Logic	scenarios/ Modules	`Modular Scenario Classes`	Domain-isolated implementations for Account Management, Lifecycle States, Role Requests, and Inspection.
API Client	sailpoint/ API Services	`HTTPS REST / OAuth 2.0 Bearer`	Token acquisition, automatic refresh, retry logic with exponential backoff, rate limiting, and log redaction.
Identity Cloud	SailPoint ISC Tenant	`v3 & Beta REST Endpoints`	Identity repository, lifecycle engine, provisioning task scheduler, and search indexing.
Target Directory	Active Directory Source	`VA / SCIM / RPC Connector`	Downstream account provisioning, account enablement/disablement, userAccountControl, and group membership.
Audit & Reports	reporting/ Subsystem	`Excel (openpyxl) / JSON / MD`	Generates 5-tab analytical Excel workbooks, Markdown executive summaries, and sanitized execution logs.

2. END-TO-END 9-STEP TESTING WORKFLOW

Step	Workflow Phase	Target API Endpoint	Operational Function
Step 0	OAuth2 Authentication	`POST /oauth/token`	Acquires Bearer token using client_id and client_secret from .env; caches in memory.
Step 1	Prompt for User	`POST /v3/search`	Prompts tester for Name, Email, or ID; lists matching test identities loaded in tenant.
Step 2	Retrieve User Details	`GET /beta/identities/{id}`	Displays clean Identity Card with profile, email, state, and primary source.
Step 3	Confirm Identity Profile	`GET /v3/identity-profiles`	Defaults to user's assigned profile (e.g. Truist Employee Profile) or allows selection.
Step 4 & 5	Inspect Lifecycle States	`GET .../lifecycle-states`	Dynamically queries profile's configured states (preHire, active, skp, ear, loa, terminated).
Step 6	Select Target Source	`GET /v3/sources`	Defaults to AD_Demo_Source; displays available Active Directory connectors.
Step 7	Choose Test Scenario	`Interactive MenuManager`	Displays 2-tier categorized menu across Account, Lifecycle, Access, and Inspection.
Step 8	Execute Scenario	`POST /v3/provisioning /set-state`	Triggers state change, direct account enable/disable/create, or role request.
Step 9	4-Tier Validation & Audit	`GET /beta/account-activities`	Polls task completion, validates live identity state, asserts AD account status, and logs report.

3. EXTENSIBLE 2-TIER MODULAR CORE ARCHITECTURE

To satisfy enterprise scalability across multiple client tenants, heterogeneous Active Directory environments, and evolving governance use cases, the framework decouples execution runtime from individual test scenarios. The architecture adheres strictly to the **Open-Closed Principle** (open for extension, closed for modification).

Core Class / File	Architectural Pattern	Technical Role & Extensibility Mechanism
`BaseScenario (core/base_scenario.py)`	**Abstract Template Method**	Abstract base class defining required scenario attributes (scenario_id, name, category_id, order) and lifecycle hooks (confirm_execution, execute). Ensures consistent signature and behavior.
`ExecutionContext (core/context.py)`	**Context Object / Facade**	Encapsulates mutable session state (identity, source, profile_id, report_generator, interactive, force_confirmed). Provides refresh_identity() helper to eliminate stale data caching.
`ScenarioRegistry (core/registry.py)`	**Service Locator / Registry**	Dynamic central catalog. Scenarios register via @register_scenario decorator. Organizes scenarios by category, sorts by execution order, and maps legacy CLI numbers (1-10) for backward compatibility.
`MenuManager (core/menu_manager.py)`	**2-Tier Hierarchical Controller**	Renders dynamic top-level category menu and sub-menus. Handles interactive selection, confirmation prompts, Step 8 execution dispatch, and Step 9 validation rendering.

BLUEPRINT: HOW TO ADD ANY FUTURE TEST SCENARIO (ZERO BLAST-RADIUS)

```
from core.base_scenario import BaseScenario
from core.context import ExecutionContext
from core.registry import register_scenario
from reporting.report_generator import TestReportItem

@register_scenario
class ParentalLeaveScenario(BaseScenario):
    scenario_id = 'lifecycle.parental_leave'
    name = 'Parental Leave - Suspension'
    category_id = 'lifecycle_management'
    expected_ad_outcome = 'Suspend AD account and disable access'
    order = 7

    def execute(self, ctx: ExecutionContext) -> TestReportItem:
        # Custom test logic here...
        return report
```

4. DOMAIN TEST SCENARIOS TAXONOMY

Category	Scenario ID	Display Name	Expected Active Directory Outcome
Account Management	`account.create`	Create Target Account	Creates and enables a new AD account on the target source via /v3/accounts.
Account Management	`account.enable`	Enable Account	Explicitly enables target AD account via /v3/accounts/{id}/enable.
Account Management	`account.disable`	Disable Account	Explicitly disables target AD account via /v3/accounts/{id}/disable.
Account Management	`account.unlock`	Unlock Account	Clears lockout flag (userAccountControl) via /v3/accounts/{id}/unlock.
Lifecycle Management	`lifecycle.active`	Active (Onboarding / Rehire)	Applies active state; triggers onboarding policy and enables standard AD access.
Lifecycle Management	`lifecycle.loa`	Leave of Absence - LOA	Applies loa state; disables AD account and suspends access while retaining attributes.
Lifecycle Management	`lifecycle.skp`	Sabbatical - SKP	Standardized project abbreviation SKP. Suspends AD access per tenant policy.

Lifecycle Management	`lifecycle.ear`	Emergency Access Removal (EAR)	High-impact security transition requiring confirmation; immediately revokes critical access.
Lifecycle Management	`lifecycle.termination`	Employment Termination	Requires explicit 'TERMINATE' prompt; revokes all entitlements and disables/removes AD account.
Lifecycle Management	`lifecycle.full_demo`	Full Lifecycle Demonstration	Sequential end-to-end execution: Active -> SKP -> LOA -> EAR -> Termination.
Access Management	`access.role_request`	Role Request (Source Validated)	Inspects role's access profiles; validates target source match before submission.
Inspection	`inspection.identity_details`	View Identity Details & Attributes	Read-only inspection of identity model, email, profile, and attributes (Image 1 view).
Inspection	`inspection.view_accounts`	View Correlated Accounts & Attributes	Summary table of all correlated accounts (AD, Delimited, IdentityNow) (Image 2 view).
Inspection	`inspection.source_attributes`	View Target Source Attributes	Direct drill-down into target AD account's raw schema attributes (Image 3 view).
Inspection	`inspection.event_attributes`	View Identity Event Attributes	Audit trail with 3 options: Last 10 events, source-filtered events, or complete history.

5. EXHAUSTIVE FILE-BY-FILE COMPONENT REFERENCE

This section documents every file in the codebase, detailing its purpose, architectural responsibility, and integration points within the framework.

Package / File Path	Primary Responsibility & Operational Purpose	Why It Is Used
`main.py`	Main CLI coordinator orchestrating the 9-step workflow from authentication to final audit report generation.	Central user interface and script entry point.
`requirements.txt`	Defines production package dependencies (requests, python-dotenv, openpyxl).	Enables single-command dependency installation via pip.
`.env.example`	Template documenting required environment variables (SAILPOINT_BASE_URL, CLIENT_ID, CLIENT_SECRET).	Allows secure credential configuration without exposing secrets.
`.gitignore`	Exclusion rules protecting secrets (.env), caches (__pycache__), and logs/reports from Git.	Prevents accidental leaks of PAT tokens or corporate audit logs.
`core/base_scenario.py`	Abstract base class BaseScenario enforcing standard attributes and execution methods.	Guarantees uniform contract for all test scenarios.
`core/context.py`	ExecutionContext runtime model maintaining identity, source, profile, and reporter references.	Avoids parameter proliferation across scenario methods.
`core/registry.py`	ScenarioRegistry and @register_scenario decorator providing dynamic discovery.	Decouples scenario registration from runner code.
`core/menu_manager.py`	MenuManager managing 2-tier hierarchical CLI navigation and scenario dispatch.	Replaces flat lists with organized category navigation.
`scenarios/account_management/create_account.py`	Tests account creation via /v3/accounts if not existing, applies active state.	Validates initial AD onboarding provisioning.
`scenarios/account_management/enable_account.py`	Explicitly invokes /v3/accounts/{id}/enable and validates enabled status.	Validates explicit account restoration/enablement.
`scenarios/account_management/disable_account.py`	Explicitly invokes /v3/accounts/{id}/disable and validates disabled status.	Validates explicit account disablement.
`scenarios/account_management/unlock_account.py`	Invokes /v3/accounts/{id}/unlock and clears AD lockout bitmask.	Validates lockout remediation on AD accounts.
`scenarios/lifecycle_management/active.py`	Applies active lifecycle state and validates account enablement.	Validates standard active employee provisioning.
`scenarios/lifecycle_management/leave_of_absence.py`	Applies loa state and asserts account suspension.	Validates temporary leave access suspension.
`scenarios/lifecycle_management/skp.py`	Applies skp sabbatical state and validates policy enforcement.	Standardized sabbatical policy testing.
`scenarios/lifecycle_management/emergency_access_removal.py`	Applies ear state with mandatory EAR-CONFIRM safety check.	Validates rapid security lockout in incident scenarios.
`scenarios/lifecycle_management/termination.py`	Applies termination state with mandatory TERMINATE prompt.	Validates deprovisioning and entitlement revocation.
`scenarios/lifecycle_management/full_lifecycle_demo.py`	Runs sequential demo: Active -> SKP -> LOA -> EAR -> Termination.	Validates complete identity lifecycle progression.
`scenarios/access_management/role_request.py`	Role search, source alignment validation, request submission, and polling.	Validates role governance and target source matching.
`scenarios/inspection/view_identity_details.py`	Displays clean Identity Card with profile and attributes (Image 1).	Read-only inspection of identity attributes.
`scenarios/inspection/view_correlated_accounts.py`	Renders table of all correlated accounts across sources (Image 2).	Read-only inspection of multi-source accounts.
`scenarios/inspection/view_source_attributes.py`	Drill-down into target AD account's raw schema attributes (Image 3).	Read-only inspection of directory attributes.
`scenarios/inspection/view_event_attributes.py`	Audit log inspection with 3 options: Last 10, source filter, and all events.	Read-only audit event log investigation.
`sailpoint/api_client.py`	Central HTTP client with automatic retry, backoff, and bearer token redaction.	Robust, centralized communication with SailPoint ISC.

`sailpoint/identity.py`	Identity search (/v3/search), detail lookups, and card view formatters.	Identity domain queries and terminal displays.
`sailpoint/identity_profiles.py`	Retrieves identity profiles and configured lifecycle states from tenant.	Dynamic retrieval of tenant governance configuration.
`sailpoint/sources.py`	Retrieves tenant sources and filters Active Directory connectors.	Identifies valid provisioning targets in tenant.
`sailpoint/accounts.py`	Account creation, direct enablement, disablement, and detail queries.	Low-level account management operations.
`sailpoint/roles.py`	Role search, access profile resolution, and source alignment check.	Verifies role targets correct directory before request.
`sailpoint/access_requests.py`	Submits access requests (/v3/access-requests) and polls status.	Governed entitlement grant orchestration.
`sailpoint/lifecycle.py`	Executes set-lifecycle-state API and captures account activity IDs.	Triggers tenant lifecycle engine evaluation.
`sailpoint/events.py`	Queries /v3/search events index, parses sources, and formats event tables.	Dedicated service powering event inspection.
`validation/identity_validation.py`	Validates live technical lifecycle state against expectation.	Tier 2 identity validation.
`validation/account_validation.py`	Validates account existence and enabled/disabled status via UAC/flags.	Tier 3 account validation.
`validation/provisioning_validation.py`	Polls /beta/account-activities and verifies downstream policy match.	Tier 1 task polling and Tier 4 policy evaluation.
`reporting/report_generator.py`	Maintains TestReportItem records; generates JSON, Markdown, and Excel.	Central audit report coordination.
`reporting/excel_report_generator.py`	Constructs 5-tab analytical Excel workbook with corporate styling.	Leadership-ready analytical reporting.
`reporting/logger.py`	Console and rotating file logger with automatic token redaction.	Sanitized, timestamped diagnostic logging.
`authentication/auth_client.py`	OAuth2 Client Credentials manager with in-memory token caching.	Secure, automated API authentication.
`config/settings.py`	Environment configuration loader resolving paths and tenant URLs.	Centralized application configuration.
`config/constants.py`	Constant mappings for lifecycle states, API endpoints, and metadata.	Eliminates hardcoded strings across codebase.
`docs/SETUP_GUIDE.md`	Autonomous machine setup guide with step-by-step terminal instructions.	Enables zero-AI setup on any fresh machine.
`docs/API_DOCUMENTATION.md`	Complete reference of all SailPoint ISC REST endpoints utilized.	Technical API integration specification.
`docs/WORKFLOW.md`	Comprehensive workflow documentation with sequence diagrams and rules.	Detailed testing process guide.

6. 4-TIER VERIFICATION PIPELINE & INSTITUTIONAL REPORTING

A critical differentiator of this framework is its **4-Tier Technical Verification Pipeline**. A test scenario is not marked as successful merely because the API returned HTTP 200 OK. It must satisfy deterministic checks across all four independent verification tiers.

Verification Tier	Target System & API	Verification Criteria & Assertion
Tier 1: Provisioning Activity Polling	`GET /beta/account-activities/{id}`	Polls backend task queue until status transitions from PENDING/IN_PROGRESS to COMPLETED. Confirms task completed without queue exceptions.
Tier 2: Real-time Live Identity State	`GET /beta/identities/{id}`	Directly queries live identity object to assert technical lifecycleState matches target (e.g. active, loa, skp, terminated).
Tier 3: Target Account Existence & Status	`GET /v3/accounts?filters=...`	Inspects target AD account record. Asserts root disabled boolean and decodes Active Directory userAccountControl (512 vs 514).
Tier 4: Downstream Policy & Outcome Evaluation	`Framework Policy Engine`	Correlates observed directory state against tenant identity profile expectation (e.g. confirming LOA resulted in AD disablement).

INSTITUTIONAL AUDIT & REPORTING ARTIFACTS

Upon session completion, the framework automatically generates three synchronized audit artifacts:

Artifact Format	File Path & Schema	Content & Analytical Purpose
Analytical Excel Workbook	`reports/test_report_.xlsx`	Corporate 5-tab report (Executive Summary, Scenario Master Log, Step-by-Step Audit, Verification Matrix, Account Inventory). Zero emojis, institutional typography, and KPI cards.
Executive Markdown Summary	`reports/test_summary_.md`	Human-readable document containing executive summary tables, verification check matrices, and failure rationales for rapid peer review.
Raw Audit JSON Data	`reports/test_report_.json`	Full machine-readable export with complete test parameters, raw API payloads, task IDs, and verification check dictionaries for CI/CD ingestion.
Sanitized Execution Log	`logs/execution.log`	Comprehensive chronological audit trail recording every HTTP request, status code, latency, and assertion, with automatic Bearer token redaction.

7. SECURITY, CREDENTIAL ISOLATION & ZERO-TRUST GOVERNANCE

ENTERPRISE SECURITY & DATA PRIVACY CONTROLS:
• **Credential Isolation**: Personal Access Tokens (PAT Client ID & Client Secret) are loaded exclusively via environment variables (.env). Credentials are never hardcoded, never stored in version control, and .env is strictly excluded by .gitignore.
• **In-Memory OAuth Token Lifecycle**: Access tokens are acquired dynamically via POST /oauth/token, cached in memory with expiration tracking, and automatically refreshed before expiry without writing tokens to disk.
• **Sensitive Data Redaction**: The custom logging engine (reporting/logger.py) inspects all outgoing requests and diagnostic messages, automatically regex-sanitizing Authorization headers, Bearer tokens, and client secrets.
• **High-Impact Guardrails**: Dangerous transitions (Employment Termination, Emergency Access Removal) mandate interactive safety confirmation ('TERMINATE' / 'EAR-CONFIRM') to prevent accidental deprovisioning of non-test identities.

Document Reference: SAILPOINT-ISC-AD-PROV-ARCH-v2.0 | Approved for Enterprise Distribution